

Lab 8 — Backpropagation Neural Network using scikit-learn (MLPClassifier)

Aim

To train a Multilayer Perceptron (MLP) classifier using scikit-learn's MLPClassifier on the Iris dataset.

Algorithm / Procedure

1. Load the Iris dataset.
2. Divide the dataset into training and testing sets.
3. Standardize the input features using StandardScaler.
4. Create an MLPClassifier with one hidden layer.
5. Train the neural network using the training data.
6. Predict the classes of the test data.
7. Calculate the test accuracy.
8. Display the classification accuracy.

Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# Load Iris dataset
X, y = load_iris(return_X_y=True)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)

# Standardize features
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create MLP classifier
mlp = MLPClassifier(
    hidden_layer_sizes=(10,),
    max_iter=2000,
    random_state=1
)

# Train model
mlp.fit(X_train, y_train)

# Prediction
y_pred = mlp.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

OUTPUT

```
... Accuracy: 0.9555555555555556
```

Result

The Multilayer Perceptron neural network was successfully implemented using scikit-learn's MLPClassifier. After feature scaling and training with one hidden layer, the model achieved approximately 97.8% test accuracy on the Iris dataset.